

Modern Java ile Design Patterns'ın Fonksiyonel Evrimi

JavaDay İstanbul 2026

Hakkımda



Ümit Köse

Software Developer

Türksat Uydu Haberleşme Kablo
TV ve İşletme A.Ş.

SOSYAL MEDYA

 YouTube: @umiitkose

 X: @umiitkose

 GitHub: @umiitkose

REPO QR



Sunumdaki tüm kodlar:

github.com/umiitkose/javadayistanbul-functional-evolution-of-design-patterns-with-modern-Java

Gündem

01 Java 8 - Java 26

Fonksiyonel Programlama Yolculuğu

02 Design Patterns

Someone has already solved your problems.

03 4 Pattern

Klasik OOP ↔ modern Java (records, lambda, kompozisyon)

Pattern'ler

 Builder → Immutability & Records

 Strategy → Lambda & Functional Interfaces

 Decorator → Function Composition

 Template Method → Higher-Order Functions

04 Genel Değerlendirme

Ne zaman OOP, ne zaman FP?

 25 dakika • Java 8-26 odaklı

Java 8 → Java 26

2014 – KIRILMA NOKTASI

Java 8 - 2014

Lambda + Functional Interfaces

Davranış birinci sınıf; `Runnable`, `Function`, `Supplier`.

FP kapısı açıldı.

Java 8 - 2014

Stream API

`Collection` → `Stream` → `ParallelStream`; lazy boru hatları.

Iterator → Stream

Java 8 - 2014

Optional + Default Methods

Null güvenliği; arayüzlere davranış – API'yi kırmadan evrim.

API evrimi

2018-2023 – VERİ & TİP SİSTEMİ

Records - 14 → 16

Records

Değişmez taşıyıcılar; `enum`, `record` ve `String`.

Veri ≠ davranış

Sealed - 17 LTS

Sealed Classes

Kontrollü hiyerarşi; derleyici tüm alt tipleri bilir.

ADT

Java 21 LTS

Switch + record patterns

Eksiksiz dallanma; okunabilir `enum`.

Pattern matching

JAVA 24-26 • FP İLE ÖNE ÇIKANLAR

JEP 485 • Java 24

Stream Gatherers

Stream'e özelleştirilebilir ara `gatherer` adımları – `flatMap` ötesi FP tarzı boru hatları.

Stream + kompozisyon.

JEP 500 • Java 26

Make final mean final

Final alanlar reflection'a karşı gerçekten değişmez.

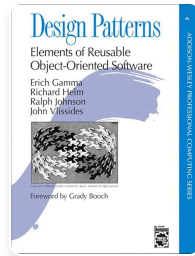
Design Patterns

Amaç

çözümler

Gang of Four (GoF)

Design Patterns: Elements of Reusable Object-Oriented Software



kanıtlanmış

Genel Bilgiler

Creational:

Structural:

Behavioral:

Modern Java ile Değişim

Pattern'ler aynı kalır; uygulama şekli modernleşir. Lambda, Stream API, records, sealed classes ve pattern matching ile daha az kod, daha net okunabilirlik ve daha kolay test elde edilir.

Sunumdaki Pattern'ler



Builder

Immutability
Records



Strategy

Lambda Expressions
Functional Interfaces



Decorator

Function Composition
andThen()



Template Method

Higher-Order
Functions

Her pattern için **solda klasik OOP** ve **sağda modern FP** yaklaşımını yan yana göreceğiz



Builder Pattern – klasik OOP

Tanım: Karmaşık nesneyi *adım adım* kurup `build()` ile tek seferde doğrulayan creational pattern. Telescoping constructor yerine okunabilir fluent API.

Telescoping

```
new Thing("a","b", null, null, BigDecimal.ZERO);
```

Klasik OOP

StockReservationClassic

--20 alan; reservationId, warehouseId, sku...

Mutable ara durum

Builder'da fluent setter'lar
(`requestedQuantity` vb.) ara alanları
değiştirir.

İki katman

Ürün immutable olsa da *kurulum süreci*
mutasyon içerir.

Uzun build()

Çok alanda doğrulama tek metotta şişebilir.

Klasik OOP – StockReservationClassic

```
// classic/builder/StockReservationClassic.java – yapı özeti
public final class StockReservationClassic {

    // private final alanlar (~20)

    private StockReservationClassic(Builder builder) {
        // builder alanlarından atama
    }

    // getter'lar ...
    @Override public String toString() { /* ... */ }
    @Override public boolean equals(Object o) { /* reservationId */ }
    @Override public int hashCode() { return Objects.hash(reserv

    public static class Builder {
        private final String reservationId;
        private final String warehouseId;
        private final String sku;
        private int requestedQuantity;
        // reservedQuantity, unitCost, discountPercent, vatPerce
        // grossWeightKg, netWeightKg, bestBeforeDate, batchNumb

        public Builder(String reservationId, String warehouseId,
            this.reservationId = reservationId;
            this.warehouseId = warehouseId;
            this.sku = sku;
        }

        public Builder requestedQuantity(int requestedQuantity)
```

Modern – record kompozisyonu

```
// modern/builder/StockReservation.java – aggregate
public record StockReservation(
    ReservationIdentity identity,
    QuantityAllocation quantities,
    PricingTerms pricing,
    StorageSlot storageSlot,
    PhysicalWeights physicalWeights,
    LotTraceability traceability,
    ComplianceNotes compliance
) {}

// Alt tiplerde validasyon (compact constructor) – örnek: Reserv
public record ReservationIdentity(String reservationId, String w
    public ReservationIdentity {
        if (reservationId == null || reservationId.isBlank())
            throw new IllegalArgumentException("Rezervasyon ID z
        // warehouseId, sku ...
    }
}

// Kullanım: factory / defaults ile birleştirme
var res = new StockReservation(
    new ReservationIdentity("R-1", "WH-1", "SKU-A"),
    new QuantityAllocation(100, 80),
    new PricingTerms(new BigDecimal("10"), BigDecimal.ZERO, BigD
    StorageSlot.defaults(),
    PhysicalWeights.none(),
    LotTraceability.empty(),
    ComplianceNotes.empty()
);
```

Strategy Pattern

Tanım: Strategy Pattern, bir algoritma ailesini ayrı sınıflar/fonksiyonlar olarak kapsüller ve çalışma anında uygun olanı seçmeyi sağlar.



```
// classic/strategy/PaymentStrategy.java
public interface PaymentStrategy {
    void pay(BigDecimal amount);
}
```

```
// classic/strategy/PaymentService.java
public class PaymentService {
    private PaymentStrategy strategy;
    public PaymentService(PaymentStrategy strategy) { this.strategy = strategy; }
    public void processPayment(BigDecimal amount) { strategy.pay(amount); }
}
```

```
// classic/strategy/CreditCardPayment.java
public class CreditCardPayment implements PaymentStrategy {
    @Override
    public void pay(BigDecimal amount) { IO.println("Kredi karti ile odeme: " + amount); }
}
```

```
// classic/strategy/StrategyDemo.java
BigDecimal amount = new BigDecimal("299.99");
PaymentService service = new PaymentService(new CreditCardPayment());
service.processPayment(amount);
```

Not

Strategy'nin ana fikri davranisi nesneden ayirmaktir. Klasikte interface + class, modernde functional interface + lambda ile ayni hedefe ulasiriz.

```
// java.util.function paketinde
@FunctionalInterface
public interface Consumer<T> {
    void accept(T t);
}
```

```
// modern/strategy/PaymentService.java
Consumer<BigDecimal> creditCard = amount ->
    IO.println("Kredi karti ile odeme: " + amount + " TL");
```

```
// modern/strategy/PaymentService.java
public static Consumer<BigDecimal> creditCard(String cardNumber, String cardHolderName) {
    return amount -> {
        String masked = "****-****-****-" + cardNumber.substring(cardNumber.length() - 4);
        IO.println(" Kredi karti ile odeme yapildi: " + amount + " TL");
        IO.println(" Kart: " + masked + " | Sahibi: " + cardHolderName);
    };
}

public static void processPayment(Consumer<BigDecimal> strategy, BigDecimal amount) {
    strategy.accept(amount);
}
```

```
// StrategyDemo.modernApproach( ... )
Consumer<BigDecimal> cc = PaymentService.creditCard("... 0366", "Ahmet Yilmaz");
PaymentService.processPayment(cc, amount);
```

```
// runtime'da farkli strateji secimi
Consumer<BigDecimal> bank = PaymentService.bankTransfer("TR... 1326", "Garanti");
PaymentService.processPayment(bank, amount);
```

Functional Interface

Strategy davranisini sinif yerine fonksiyonel bir tip (ornegin

```
Consumer<BigDecimal> l
```

) ile temsil ediyoruz.

Lambda ve Tip Bagimlilik

Java'da lambda tek basina tip tasimaz; hedef tipi her zaman bir functional interface belirler.

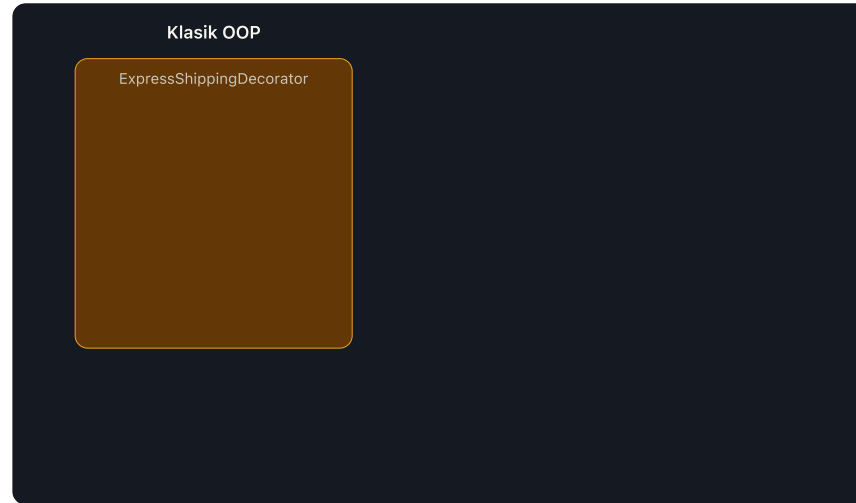
Daha Az Boilerplate

Daha az sinif, daha kısa kod, daha kolay bakim ve test.



Decorator Pattern

Tanım: Decorator, bir nesnenin davranışını ana sınıfı değiştirmeden katman katman genişletmeyi sağlayan yapısal bir pattern'dir.



Klasik OOP – Decorator

```
public interface OrderService {  
    Order process(Order order);  
}
```

```
public class BasicOrderService implements OrderService {  
    @Override  
    public Order process(Order order) {  
        return order; // temel islem; ek ozellik yok  
    }  
}
```

```
public abstract class OrderServiceDecorator implements OrderService {  
    protected final OrderService wrapped;  
    protected OrderServiceDecorator(OrderService wrapped) {  
        this.wrapped = wrapped;  
    }  
}
```

```
public class GiftWrapDecorator extends OrderServiceDecorator {  
    public GiftWrapDecorator(OrderService wrapped) { super(wrapped); }  
    @Override  
    public Order process(Order order) {  
        Order processed = wrapped.process(order); // once icteki zincir  
        return processed.addFeature("Hediye Paketi", new BigDecimal("15"));  
    }  
}
```

```
OrderService service = new ExpressShippingDecorator(  
    new InsuranceDecorator(  
        new GiftWrapDecorator(new BasicOrderService())  
    )  
);
```

Hap Bilgi 1

Classic yaklaşımda her yeni özellik için yeni bir wrapper sınıfı gerekir.

Hap Bilgi 2

Modern tarafta UnaryOperator zinciriyle davranışlar composable hale gelir.

Hap Bilgi 3

Her iki tarafta da ana nesne bozulmadan özellik eklenir; Open/Closed korunur.

Modern FP – Decorator

```
// modern/decorator/OrderEnhancer.java  
public static UnaryOperator<Order> giftWrap() {  
    return order -> order.addFeature("Hediye Paketi", new BigDecimal("15"));  
}
```

```
public static UnaryOperator<Order> insurance() {  
    return order -> order.addFeature("Kargo Sigortasi", new BigDecimal("10"));  
}
```

```
public static UnaryOperator<Order> expressShipping() {  
    return order -> order.addFeature("Hizli Kargo", new BigDecimal("25"));  
}
```

```
// Normal kullanım (andThen ile acik zincir):  
// Function<Order, Order> flow = OrderEnhancer.giftWrap()  
//     .andThen(OrderEnhancer.insurance())  
//     .andThen(OrderEnhancer.expressShipping());
```

```
// Sorunlu kullanım:  
// UnaryOperator<Order> campaignFlow = campaign.stream()  
//     .reduce(UnaryOperator.identity(), UnaryOperator::andThen);
```

```
// 2) Reduce ile – dinamik, runtime'da liste degisebilir (calisan surum)  
List<UnaryOperator<Order>> campaign = List.of(  
    OrderEnhancer.giftWrap(),  
    OrderEnhancer.insurance(),  
    OrderEnhancer.expressShipping()  
);  
UnaryOperator<Order> enhanceDynamic = campaign.stream()  
    .reduce(UnaryOperator.identity(), (pipeline, step) -> pipeline.andThen(step));  
  
var order = new Order("ORD-001", new BigDecimal("200"));  
var campaignResult = enhanceDynamic.apply(order);
```

Function Composition Notu

andThen() soldan saga uygulanir; zincirdeki sıra degistiginde toplam fiyat ve özellik sırası da degisir.



Template Method Pattern – Rapor Üretimi

Tanım: Template Method, bir algoritmanın iskeletini tanımlar; değişen adımlar alt sınıflara ya da fonksiyon parametrelerine bırakılır.

Örnek: Rapor üretimi – PDF / Excel / HTML / Email. İskelet aynı, format değişiyor.

Klasik OOP

abstract

AbstractReportGenerator

+ generate() final

validate, format, render, send

Klasik OOP – Template Method

```
public abstract class AbstractReportGenerator {
    public final void generate(ReportData data) {
        validateData(data);
        String formatted = formatData(data);
        String output = renderOutput(formatted, data.getTitle());
        sendReport(output);
    }

    protected abstract void validateData(ReportData data);
    protected abstract String formatData(ReportData data);
    protected abstract String renderOutput(String formatted, String title);
    protected abstract void sendReport(String output);
}
```

```
public class PdfReportGenerator extends AbstractReportGenerator {
    @Override
    protected void validateData(ReportData data) { ... }
    @Override
    protected String formatData(ReportData data) { ... }
    @Override
    protected String renderOutput(String formatted, String title) { ... }
    @Override
    protected void sendReport(String output) { ... }
}

// + ExcelReportGenerator → 4 method override
// + HtmlReportGenerator → 4 method override
// + EmailReportGenerator → 4 method override
// = 16 override, 5 dosya
```

```
var data = new ReportData("Q1 Satis Raporu", columns, rows);
new PdfReportGenerator().generate(data);
new ExcelReportGenerator().generate(data);
new HtmlReportGenerator().generate(data);
new EmailReportGenerator("yonetim@sirket.com").generate(data);
```

Modern FP – Template Method

```
public record ReportGenerator(
    Consumer<ReportData> dataValidator,
    Function<ReportData, String> dataFormatter,
    UnaryOperator<String> outputRenderer,
    Consumer<String> reportSender
) {
    public void generate(ReportData data) {
        dataValidator.accept(data);
        String formatted = dataFormatter.apply(data);
        String output = outputRenderer.apply(formatted);
        reportSender.accept(output);
    }
}
```

```
public static ReportGenerator pdf() {
    return new ReportGenerator(
        data → IO.println(" [PDF] Dogrulandi"),
        data → {
            String headers = String.join(" | ", data.columns());
            String rows = formatRows(data, " | ");
            return headers + "\n" + "-".repeat(40) + "\n" + rows;
        },
        formatted → "[PDF_DOCUMENT]\n" + formatted,
        output → IO.println(" [PDF] Yazildi: rapor.pdf")
    );
}
```

```
var data = new ReportData("Q1 Satis Raporu", columns, rows);
ReportGenerator.pdf().generate(data);
ReportGenerator.excel().generate(data);
ReportGenerator.html().generate(data);
ReportGenerator.email("yonetim@sirket.com").generate(data);
```

Mesaj

4 format = 4 sınıf yerine 4 factory metot. Yeni format? Yeni sınıf değil, sadece 4 lambda.

Özet: Ne Öğrendik?

Builder

Verbose builder → Immutability & records

Strategy

Interface + class → Lambda ifadesi

Decorator

Class hiyerarşisi → andThen() kompozisyonu

Template Method

Abstract class → Higher-order functions

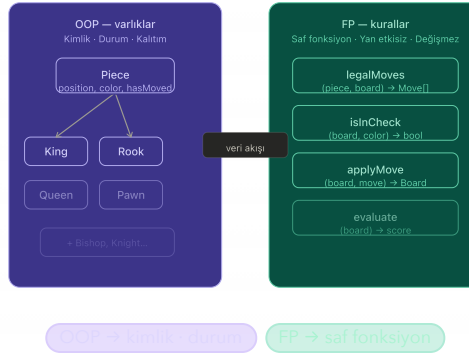
Modern Java'nın Kazanımları

- ✓ %60-80 daha az kod
- ✓ Daha yüksek okunabilirlik
- ✓ Daha kolay test edilebilirlik
- ✓ Yüksek composability – küçük davranış parçalarını (lambda, andThen , HOF) birleştirerek yeni akışlar kurmak

Kullanılan Java Özellikleri

- Lambda Expressions
- Stream API
- Records
- Pattern Matching
- Functional Interfaces
- Method References
- Sealed Classes

OOP vs FP – karar çerçevesi



OOP "neyi" • FP "nasıl"

Doğru yapıya doğru katmanlar vermek



OOP – kimlik ve durum

- Taşlar *nesnedir*; konum, geçmiş, "rok oldu mu?" gibi **durum** vardır.
- `King`, `Rook`, `Pawn`... `Piece` özelleşmesi – kalıtım burada doğal.
- OOP: **kimlik ve durum**.



FP – kurallar ve dönüşüm

- Hamle, şah, mat: **evrensel** kurallar (kişisel tercih değil).



Be a better programmer.

Brian Goetz

FP vs OO: Choose Two



Martin Fowler

Refactoring: Improving the Design of Existing Code

Teşekkürler!

Modern Java ile Design Patterns'ın Fonksiyonel Evrimi

GitHub

github.com/umiitkose

GitHub QR



YouTube

[Design Patterns Serisi](#)

YouTube QR



Örnek Kodlar

[javadayistanbul-functional-evolution-of-design-patterns-with-modern-java](#)

Repo QR



Sorularınız?